Linguaggi di Programmazione

Manipolazione di grammatiche.

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Quali manipolazioni sono utili?

Per avere parser più efficienti (ovvero con minor nondeterminismo), è necessario manipolare le grammatiche:

- eliminare le produzioni ϵ (del tipo $A \to \epsilon$), che sono poco adatte al bottom-up parsing.
- eliminare le produzioni unitarie (del tipo $A \to B$), che possono creare dei cicli $A \Rightarrow^+ A$.
- eliminare i simboli inutili, ovvero quei simboli che non sono raggiungibili dal simbolo iniziale S (così come quei nonterminali che non generano parole).
- eliminare la ricorsione sinistra (del tipo $A \to A\alpha$), perché inadatta al top-down parsing.
- fattorizzare la grammatica, per ottenere grammatiche con meno nondeterminismo nel top-down parsing.

Eliminare le produzioni ϵ

Algoritmo per eliminare le produzioni ϵ

Input: G libera con produzioni ϵ (del tipo $A \to \epsilon$)
Output: G' libera, senza produzioni ϵ, tale che $L(G') = L(G) \setminus \{\epsilon\}$
Osservazione: Se $\epsilon \in L(G)$ e vogliamo G'' senza produzioni ϵ tale che $L(G) = L(G'')$, basta prendere la grammatica $G' = (NT, T, S, R')$ (data in output dall'algoritmo) e definire $G'' = G' \cup \{S' \to \epsilon \mid S\}$, quindi $G'' = (NT \cup \{S'\}, T, S', R' \cup \{S' \to \epsilon \mid S\})$.

- $A \in NT$ è detto **simbolo annullabile** se $A \Rightarrow^+ \epsilon$.
- l'insieme dei simboli annullabili $N(G) = \{A \in NT | A \Rightarrow^+ \epsilon\}$ viene calcolato induttivamente come segue:
  - $N_0(G) = \{A \in NT \mid A \to \epsilon \in R\}$
  - $N_{i+1}(G) = N_i(G) \cup \{B \in NT \mid B \to C_1 ... C_k \in R \wedge C_1, ..., C_k \in N_i(G)\}$
- Abbiamo che $N_i(G) \subseteq N_{i+1}(G)$, perché aggiungo qualcosa ad ogni passo, e che $\exists \underline{i}$ tale che $N_{\underline{i}}(G) = N_{\underline{i}+1}(G)$, perché NT è finito.
- L'insieme $N(G) = N_{\underline{i}}(G)$ è esattamente l'insieme di tutti i simboli annullabili.

Algoritmo per eliminare le produzioni ϵ (2)

Una volta calcolato $N(G)$ per $G = (NT, T, S, R)$, costruiamo la grammatica $G' = (NT, T, S, R')$ dove per ogni produzione $A \to \alpha \in R$ con $\alpha \neq \epsilon$, in cui occorrono i simboli annullabili $Z_1, ..., Z_k$, mettiamo in R' tutte le produzioni $A \to \alpha'$ dove α' si ottiene da α cancellando tutti i possibili sottoinsiemi di $Z_1, ..., Z_k$ ad eccezione del caso in cui $\alpha' = \epsilon$.

- Dato che parto da produzioni $A \to \alpha \in R$ con $\alpha \neq \epsilon$, in G' non mettiamo produzioni $A \to \epsilon \in R$.
- Dato che le nuove produzioni $A \to \alpha'$ sono tali che $\alpha' \neq \epsilon$, in G' non introduciamo mai produzioni del tipo $A \to \epsilon$.

Proposizione: Data una grammatica libera G, la grammatica G' determinata dall'algoritmo sopra non ha ϵ-produzioni e $L(G') = L(G) \setminus \{\epsilon\}$.

Esempio

Sia G

$$S \to AB$$
$$A \to aAA \mid \epsilon$$
$$B \to bBB \mid \epsilon$$

i cui simboli annullabili sono $N(G) = \{A, B, S\}$. Consideriamo la prima produzione $S \to AB \in R$. Dobbiamo considerare 4 casi (tutti i modi in cui possiamo cancellare i due simboli annullabili):

NT cancellati	*produzione ottenuta*
$\emptyset \Rightarrow$	$S \to AB$
$\{B\} \Rightarrow$	$S \to A$
$\{A\} \Rightarrow$	$S \to B$
$\{A, B\} \Rightarrow$	~~$S \to \epsilon$~~ non la mettiamo in R'

Quindi la grammatica G' contiene queste produzioni: $S \to AB \mid A \mid B$.

Esempio (2)

Consideriamo $A \to aAA \in R$. Dobbiamo considerare 4 casi:

$$\begin{aligned} \emptyset \Rightarrow & \qquad A \to aAA \\ \{A\} \Rightarrow & \qquad A \to aA \\ \{A\} \Rightarrow & \qquad A \to aA \text{ duplicata} \\ \{A, A\} \Rightarrow & \qquad A \to a \end{aligned}$$

Quindi G' contiene $A \to aAA \mid aA \mid a$.

Ora consideriamo $B \to bBB$ e, allo stesso modo di sopra, otteniamo $B \to bBB \mid bB \mid b$. Le due ϵ-produzioni di G, ovvero $A \to \epsilon$ e $B \to \epsilon$, sono state ignorate.
La grammatica G' risultante è:

$$S \to AB \mid A \mid B$$
$$A \to aAA \mid aA \mid a$$
$$B \to bBB \mid bB \mid b$$

Esempio (3)

Nota che $\epsilon \in L(G)$ - infatti $S \Rightarrow AB \Rightarrow B \Rightarrow \epsilon$ – ma $\epsilon \notin L(G')$.

Se vogliamo una grammatica G'' tale che $L(G) = L(G'')$, possiamo modificare la G' in questo modo:

$$S' \rightarrow \epsilon \mid S$$

$$S \text{ come per } G'$$

La grammatica G'' ammette una sola produzione ϵ iniziale (per generare la stringa ϵ). La grammatica G'' è una grammatica libera in senso stretto (come da definizione di gerarchia di Chomsky).
L'algoritmo descritto trasforma una grammatica libera G in una grammatica libera in senso stretto G''.
N.B.: l'algoritmo descritto può introdurre produzioni unitarie!

Eliminare le produzioni unitarie

Come calcolare le coppie unitarie?

- Produzioni unitaria: $A \to B$ con $A, B \in NT$.
- Coppia unitaria: (A, B) tale che $A \Rightarrow^* B$ usando solo produzioni unitarie.

Calcoliamo l'insieme delle **coppie unitarie** induttivamente come segue:

- $U_0(G) = \{(A, A) \mid A \in NT\}$
- $U_{i+1}(G) = U_i(G) \cup \{(A, C) \mid (A, B) \in U_i(G) \wedge B \to C \in R\}$

Abbiamo che $U_i(G) \subseteq U_{i+1}(G)$, perché aggiungo qualcosa ad ogni passo, e che $\exists \underline{i}$ tale che $U_{\underline{i}}(G) = U_{\underline{i}+1}(G)$, perché NT è finito.
L'insieme $U(G) = U_{\underline{i}}(G)$ è detto insieme di tutte le coppie unitarie della grammatica G.

Algoritmo per eliminare le produzioni unitarie

Algoritmo: data $G = (NT, T, S, R)$ libera, costruiamo la grammatica $G' = (NT, T, S, R')$ dove, per ogni $(A, B) \in U(G)$ e per ogni $B \to \alpha \in R$ non unitaria, mettiamo in R' la produzione $A \to \alpha$.
Osservazione: poiché per ogni $A \in NT$, la coppia $(A, A) \in U(G)$, R' contiene tutte le produzioni non unitarie di R e in aggiunta un po' di altre.

Teorema
Sia $G = (NT, T, S, R)$ libera e sia $U(G)$ l'insieme delle sue coppie unitarie. Sia $G' = (NT, T, S, R')$ la grammatica ottenuta dall'algoritmo sopra. Allora G' non ha produzioni unitarie e $L(G) = L(G')$.

Esempio

Sia G la grammatica non ambigua delle espressioni aritmetiche (che però ha produzioni unitarie):

$$E \to E + T \mid T$$
$$T \to T * A \mid A$$
$$A \to a \mid b \mid (E)$$

Abbiamo che

$$U_0(G) = \{(E, E), (T, T), (A, A)\}$$
$$U_1(G) = U_0(G) \cup \{(E, T), (T, A)\}$$
$$U_2(G) = U_1(G) \cup \{(E, A)\} = U_3(G) = U(G)$$

Quindi per $U_0(G)$, la grammatica G' contiene

$$E \to E + T$$
$$T \to T * A$$
$$A \to a \mid b \mid (E)$$

Esempio (2)

In aggiunta:

$$E \to T * A \qquad \texttt{perché } (E, T) \in U_1(G)$$
$$T \to a \mid b \mid (E) \qquad \texttt{perché } (T, A) \in U_1(G)$$
$$E \to a \mid b \mid (E) \qquad \texttt{perché } (E, A) \in U_2(G)$$

Quindi, riassumendo, la grammatica G' senza produzioni unitarie equivalente a G è

$$E \to E + T \mid T * A \mid a \mid b \mid (E)$$
$$T \to T * A \mid a \mid b \mid (E)$$
$$A \to a \mid b \mid (E)$$

Altro esempio

Consideriamo la seguente grammatica G:

$$S \to A$$
$$A \to a \mid B$$
$$B \to bS$$

L'insieme $U(G)$ delle coppie unitarie è $\{(S,S),(A,A),(B,B),$
$(S,A),(A,B),(S,B)\}$.
Quindi, la grammatica G' senza produzioni unitarie equivalente a G è

$$S \to a \mid bS$$
$$A \to a \mid bS$$
$$B \to bS$$

N.B.: La grammatica G' è ridondante, perché i simboli nonterminali A e B non servono, sono inutili.

L'algoritmo descritto può introdurre simboli inutili!

Eliminare i simboli inutili

Quali sono i simboli utili?

Un simbolo $X \in T \cup NT$ è

- un **generatore** sse $\exists w \in T^*$ con $X \Rightarrow^* w$
- **raggiungibile** sse $S \Rightarrow^* \alpha X \beta$ per qualche $\alpha, \beta \in (T \cup NT)^*$
- **utile** sse è generatore e raggiungibile.

Ovvero, X è utile se $S \Rightarrow^* \alpha X \beta \Rightarrow^* w \in L(G)$, cioè se X compare in almeno una derivazione di una stringa $w \in L(G)$.

Come possiamo eliminare i simboli inutili, cioè quelli che non sono generatori o non sono raggiungibili da S?

Esempio

Sia G

$$S \to AB \mid a$$
$$B \to b$$

Poiché $a \Rightarrow^* a$, $b \Rightarrow^* b$, $B \Rightarrow^* b$ e $S \Rightarrow^* a$, i simboli generatori sono $\{S, B, a, b\}$. Manca A, e quindi eliminiamo le produzioni che coinvolgono il simbolo A. La grammatica risultante G' è:

$$S \to a$$
$$B \to b$$

Ora B non è raggiungibile da S, per cui possiamo eliminarlo. La grammatica risultante G'' è:

$$S \to a$$

che è equivalente a G, ma non contiene simboli inutili.

L'ordine dei due passi è importante!

- Prima elimino i non-generatori, e
- poi i non-raggiungibili.

Se inverto l'ordine, allora può capitare che non elimino tutti i simboli inutili.

Se consideriamo la G dell'esempio precedente

$$S \to AB \mid a$$

$$B \to b$$

e prima elimino in non raggiungibili, non vado a modificare la grammatica, perché tutti i simboli sono raggiungibili da S. Se poi vado ad eliminare i non generatori, elimino A ed ottengo

$$S \to a$$

$$B \to b$$

che però contiene i simbol inutili B e b.

Come calcolare i generatori?

X è un generatore sse $\exists w \in T^*$ tale che $X \Rightarrow^* w$. Posso calcolare l'insieme $G(G)$ dei generatori della grammatica G induttivamente:

1. $G_0(G) = T$
2. $G_{i+1}(G) = G_i(G) \cup$
$$\{B \in NT \mid B \to C_1 ... C_k \in R \wedge C_1, ..., C_k \in G_i(G)\}$$

Osservazione: nel passo 2) è contemplato anche il caso $k = 0$: se $B \to \epsilon$, allora B è un generatore.

Abbiamo che $G_i(G) \subseteq G_{i+1}(G)$, perché aggiungo qualcosa ad ogni passo, e che $\exists \underline{i}$ tale che $G_{\underline{i}}(G) = G_{\underline{i}+1}(G)$, perché $T \cup NT$ è finito.
L'insieme $G(G) = G_{\underline{i}}(G)$ è esattamente l'insieme di tutti i simboli generatori di G.

Come calcolare i raggiungibili?

X è un raggiungibile sse $\exists \alpha, \beta \in (NT \cup T)^*$ tali che $S \Rightarrow^* \alpha X \beta$. Posso calcolare l'insieme $R(G)$ dei raggiungibili della grammatica G induttivamente come segue:

$$R_0(G) = \{S\}$$

$$R_{i+1}(G) = R_i(G) \cup \bigcup_{A \in R_i(G) \wedge A \to X_1 \ldots X_k \in R} \{X_1, \ldots, X_k\}$$

Osservazione: Abbiamo che $R_i(G) \subseteq R_{i+1}(G)$, perché aggiungo qualcosa ad ogni passo, e che $\exists \underline{i}$ tale che $R_{\underline{i}}(G) = R_{\underline{i}+1}(G)$, perché $T \cup NT$ è finito.
L'insieme $R(G) = R_{\underline{i}}(G)$ è esattamente l'insieme di tutti i simboli raggiungibili di G.

Come rimuovere i simboli inutili?

1. Prima elimino tutti i non-generatori (e tutte le produzioni che usano almeno uno di questi).
2. Poi, nella nuova grammatica, elimino tutti i non-raggiungibili (e tutte le produzioni che usano almeno uno di questi).

Teorema
Sia $G = (NT, T, S, R)$ libera tale che $L(G) \neq \emptyset$.

- Sia G_1 la grammatica che si ottiene da G eliminando tutti i simboli che non appartengono a $G(G)$, e tutte le produzioni che usano almeno uno di questi.
- Sia G_2 la grammatica che si ottiene da G_1 eliminando tutti i simboli che non appartengono a $R(G_1)$, e tutte le produzioni che usano almeno uno di questi.

Allora G_2 non ha simboli inutili e $L(G_2) = L(G)$.

N.B.: Dato che $L(G) \neq \emptyset$, S è un generatore e rimarrà in G_1 e in G_2.

Come rimuovere i simboli inutili? (2)

Dimostrazione: Per dimostrare che $L(G_2) = L(G)$, andiamo a dimostrare le due inclusioni $L(G_2) \subseteq L(G)$ e $L(G) \subseteq L(G_2)$.

- $L(G_2) \subseteq L(G)$: ovvio, perché ogni produzione di G_2 è anche una produzione di G.
- $L(G) \subseteq L(G_2)$: dobbiamo dimostrare che se $S \Rightarrow_G^* w$ allora $S \Rightarrow_{G_2}^* w$. Ma ogni simbolo usato nella derivazione $S \Rightarrow_G^*$ è ovviamente sia raggiungibile che generatore! Allora quella derivazione è anche una derivazione per G_2.

c.v.d.

Esempio

Sia G la seguente grammatica:

$S \to AB \mid aC \qquad A \to a \qquad B \to bB$
$C \to b \mid AC \qquad D \to a \mid aS$

L'insieme dei generatori è $G(G) = \{a, b, A, C, D, S\}$. Solo B non è generatore. Allora G_1 risulta

$S \to aC \qquad A \to a$
$C \to b \mid AC \qquad D \to a \mid aS$

L'insieme dei raggiungibili è $R(G) = \{S, a, C, b, A\}$, quindi D non è raggiungibile. Allora G_2 risulta

$S \to aC \qquad A \to a \qquad C \to b \mid AC$

che non presenta simboli inutili ed è equivalente a G, ovvero $L(G_2) = L(G)$.

È buona norma ....

Se, nel semplificare la grammatica G, seguiamo questo ordine:

1. prima eliminiamo le ϵ-produzioni, poi
2. eliminiamo le produzioni unitarie (quindi i cicli), infine
3. eliminiamo i simboli inutili,

allora la grammatica risultante è garantita non avere ϵ-produzioni, né produzioni unitarie, né simboli inutili, ed essere equivalente a quella di partenza!

N.B.: L'ordine è importante!

- Infatti, abbiamo visto che durante la fase di eliminazione delle ϵ-produzioni, potremmo introdurre produzioni unitarie: le ϵ-produzioni vanno eliminate prima di procedere all'eliminazione delle produzioni unitarie.
- Similmente, abbiamo visto che durante la fase di eliminazione delle produzioni unitarie, potremmo introdurre simboli inutili: le produzioni unitarie vanno rimosse prima di eliminare i simboli inutili.

Esempio

Sia G la seguente grammatica:

$$S \to aAa \qquad A \to C \qquad C \to S \mid \epsilon$$

Per togliere le ϵ-produzioni, dobbiamo calcolarci l'insieme dei simboli annullabili: $N(G) = \{C, A\}$. La grammatica risultante G' è:

$$S \to aAa \mid aa \qquad A \to C \qquad C \to S$$

dove abbiamo tolto la produzione $C \to \epsilon$ di G, e non abbiamo introdotto la produzione $A \to \epsilon$ che verrebbe generata da $A \to C$ cancellando C.

Esempio (2)

Per togliere le produzioni unitarie, dobbiamo calcolarci l'insieme delle coppie unitarie: $U(G') = \{(S,S),(A,A),(C,C),(A,C),(C,S),(A,S)\}$. La grammatica risultante G'' è:

$S \to aAa \mid aa \qquad A \to aAa \mid aa \qquad C \to aAa \mid aa$

dove $S \to aAa \mid aa$ perché $(S,S) \in U(G')$, $A \to aAa \mid aa$ perché $(A,S) \in U(G')$, e $C \to aAa \mid aa$ perché $(C,S) \in U(G')$.

Per rimuovere i simboli inutili, dobbiamo calcolarci $G(G'') = \{a,S,A,C\}$ (tutti sono generatori) e $R(G'') = \{S,a,A\}$, ovvero C non è raggiungibile e quindi va rimosso. La grammatica risultante G''' è:

$S \to aAa \mid aa \qquad A \to aAa \mid aa$

che non ha ϵ-produzioni, né produzioni unitarie, né simboli inutili.

Forme normali di grammatiche libere da contesto

Forme normali di grammatiche libere

- Forma normale di Chomsky
- Forma normale di Greibach

Garantiscono proprietà particolari alle grammatiche.

Forma normale di Chomsky
Tutte le sue produzioni sono della forma $A \to BC$ o $A \to a$ o $S \to \epsilon$.
La produzione $S \to \epsilon$ può apparire solo se $\epsilon \in L(G)$, nel qual caso S non può mai comparire nel corpo di una produzione.

- Se G libera è in forma normale di Chomsky, allora non ha ϵ-produzioni e non ha produzioni unitarie.
- Ogni grammatica libera G può essere trasformata in un'equivalente G' in forma normale di Chomsky.

Forma normale di Greibach
Tutte le sue produzioni sono della forma $A \to a\alpha$ con $\alpha \in NT^*$ (sequenza, eventualmente vuota, di nonterminali).
Può esserci la produzione $S \to \epsilon$ solo se $\epsilon \in L(G)$, nel qual caso S non può comparire nel corpo di nessun altra produzione.

Proprietà della forma normale di Greibach

- Se G libera è in forma normale di Greibach, allora non ha ϵ-produzioni e non ha produzioni unitarie.
- In più, non è ricorsiva sinistra (problema molto grave per top-down parsing).
- Ogni grammatica libera G può essere trasformata in un'equivalente G' in forma normale di Greibach.
- Dato che l'uso di una produzione introduce esattamente un terminale nella forma sentenziale, una stringa di lunghezza n ha una derivazione di esattamente n passi!
- Inoltre, se, a partire da tale forma normale, costruiamo un PDA in modo top-down in modo furbo (cioè sostituendo la mossa espandi $(q, a\alpha) \in \delta(q, \epsilon, A)$ con l'equivalente $(q, \alpha) \in \delta(q, a, A)$), otteniamo un PDA senza mosse ϵ, dimostrando così che i PDA senza mosse ϵ possono riconoscere tutti i linguaggi liberi.

N.B.: i DPDA senza ϵ-transizioni riconoscono un sottoinsieme proprio dei linguaggi liberi deterministici. Questo accade perché, senza ϵ-transizioni, l'automa non può compiere operazioni "invisibili", ovvero perde la capacità di accettare linguaggi che richiedono di svuotare (anche solo parzialmente) la pila senza consumare caratteri dell'input. (Prova con $L = \{a^n b^m c a^n \mid n, m > 0\} \cup \{a^n b^m d b^m \mid n, m > 0\}$.)

Eliminare la ricorsione sinistra

Eliminare la ricorsione sinistra

Problema per parser top-down:

- Una produzione è ricorsiva sinistra se è del tipo $A \to A\alpha$; tali produzioni generano ricorsione sinistra immediata.
- Una grammatica G è ricorsiva sinistra se per qualche $A \in NT, \alpha \in (T \cup NT)^*$ si ha che $A \Rightarrow^+ A\alpha$.

Come rimuovere la ricorsione sinistra **immediata**?

$$A \to A\alpha_1 \mid ... \mid A\alpha_n \mid \beta_1 \mid ... \mid \beta_m$$

Queste produzioni possono essere rimpiazzate da

$$A \to \beta_1 A' \mid ... \mid \beta_m A'$$
$$A' \to \alpha_1 A' \mid ... \mid \alpha_n A' \mid \epsilon$$

Se nella grammatica originale abbiamo la derivazione

$$A \Rightarrow A\alpha_{i1} \Rightarrow ... \Rightarrow A\alpha_{ik}...\alpha_{i2}\alpha_{i1} \Rightarrow \beta_i\alpha_{ik}...\alpha_{i2}\alpha_{i1}$$

nella nuova grammatica abbiamo la derivazione equivalente $A \Rightarrow \beta_i A' \Rightarrow \beta_i\alpha_{ik}A' \Rightarrow ... \Rightarrow \beta_i\alpha_{ik}...\alpha_{i2}A' \Rightarrow \beta_i\alpha_{ik}...\alpha_{i2}\alpha_{i1}A' \Rightarrow \beta_i\alpha_{ik}...\alpha_{i2}\alpha_{i1}$

Esempio

Consideriamo G con ricorsione immediata:

$$A \to Aa \mid b$$

Seguendo il procedimento, la grammatica G' equivalente senza ricorsione immediata è:

$$A \to bA'$$
$$A' \to aA' \mid \epsilon$$

Ora consideriamo G con ricorsione immediata:

$$A \to Ab \mid Ac \mid d$$

Seguendo il procedimento, la grammatica G' equivalente senza ricorsione immediata è:

$$A \to dA'$$
$$A' \to bA' \mid cA' \mid \epsilon$$

Domanda: Posso rimuovere la ricorsione sinistra da $G = S \to Sb$?

Ricorsione sinistra non immediata

Consideriamo G:

$$S \to Ba \mid b$$
$$B \to Bc \mid Sc \mid d$$

In G c'è ricorsione sinistra immediata ($B \to Bc$), ma anche non immediata ($S \Rightarrow Ba \Rightarrow Sca$).

Algoritmo per rimuovere ricorsione sinistra (anche non immediata):

- input: G libera senza ϵ-produzioni, senza produzioni unitarie, ma con ricorsione sinistra non immediata
- output: G' libera, senza ricorsione sinistra, ma può avere ϵ-produzioni

Assumiamo che i nonterminali in $NT = \{A_1, ..., A_n\}$ siano ordinati secondo un ordine fissato.

Algoritmo per rimuovere ricorsione sinistra non immediata

for $i := 1$ **to** n **do**
    for $j := 1$ **to** $i - 1$ **do**
        sostituisci ogni produzione $A_i \to A_j\alpha$
        con $A_i \to \beta_1\alpha \mid \cdots \mid \beta_k\alpha$,
        dove $A_j \to \beta_1 \mid \cdots \mid \beta_k$ sono le produzioni correnti di A_j;
    end for
    elimina la ricorsione sinistra immediata su A_i;
end for

Applichiamo l'algoritmo all'esempio

$$S \to Ba \mid b \qquad\qquad B \to Bc \mid Sc \mid d$$

- per `i = 1`, cioè per S il ciclo interno (j da 1 a 0) non viene eseguito; poiché non c'è ricorsione immediata per S, non faccio niente!
- per `i = 2`, cioè per $A_i = B$. Il ciclo interno si esegue solo per $A_j = A_1 = S$. Allora la produzione $B \to Sc$ viene rimpiazzata con $B \to Bac \mid bc$. Ora le produzioni per B sono $B \to Bc \mid Bac \mid bc \mid d$ dalle quali dobbiamo eliminare la ricorsione immediata. Il risultato è

$$B \to bcB' \mid dB' \qquad\qquad B' \to cB' \mid acB' \mid \epsilon$$

Ovvero la grammatica risultante è

$$S \to Ba \mid b$$
$$B \to bcB' \mid dB'$$
$$B' \to cB' \mid acB' \mid \epsilon$$

Perchè l'algoritmo funziona?

- L'obiettivo dell'algoritmo è che, alla fine, ogni produzione del tipo $A_i \to A_k\alpha$ sia tale che $i < k$, in modo che sia impossibile avere ricorsione sinistra non immediata.
- Quando $i = 1$, l'unica cosa che viene fatta è la rimozione dell'eventuale ricorsione sinistra immediata per A_1. Al termine, se $A_1 \to A_k\alpha$, sicuramente $1 < k$.
- All'i-esima iterazione del for esterno, tutti i nonterminali A_m, con $m < i$, hanno la proprietà desiderata. Ora il ciclo for interno aumenta progressivamente l'indice del nonterminale in prima posizione, finché, al termine del ciclo ($j = i - 1$), avremo che ogni produzione $A_i \to A_k\alpha$ sia tale che $i \leq k$. Rimuovendo infine l'eventuale ricorsione sinistra immediata, siamo sicuri che ogni produzione $A_i \to A_k\alpha$ sia tale che $i < k$.
- Quindi al termine ($i = n$), avremo che ogni produzione $A_i \to A_k\alpha$ sia tale che $i < k$, garantendo l'impossibilità di creare ricorsione sinistra non immediata.

Perchè G in input non ha ϵ-produzioni?

Consideriamo, per esempio, questa grammatica con una ϵ-produzione:

$$S \to AS \mid a$$
$$A \to \epsilon$$

Sia $S = 1, A = 2$ (nota il ciclo $S \Rightarrow AS \Rightarrow S$, per cui c'è ricorsione non immediata).
Cosa fa l'algoritmo?

- per `i = 1, j := 1..0` l'algoritmo non fa niente, perché non c'è ricorsione immediata su S;
- per `i = 2, j = 1` ancora niente, perché non c'è nessuna produzione del tipo $A \to S\alpha$.

Quindi la grammatica non viene modificata e presenta ancora ricorsione sinistra non immediata!

Perchè G in input non ha produzioni unitarie?

Consideriamo, per esempio, questa grammatica con due produzioni unitarie:

$$S \to A \mid a$$
$$A \to S$$

Sia $S = 1, A = 2$ (nota il ciclo $S \Rightarrow A \Rightarrow S$, per cui c'è ricorsione non immediata).
Cosa fa l'algoritmo?

- per `i = 1, j := 1..0` l'algoritmo non fa niente, perché non c'è ricorsione immediata su S;
- ma per `i = 2, j = 1` ho

$$S \to A \mid a$$
$$A \to A \mid a$$

Perchè G in input non ha produzioni unitarie?

$$S \to A \mid a$$
$$A \to A \mid a$$

per cui c'è ricorsione immediata su A, che si potrebbe eliminare come segue:

$$S \to A \mid a$$
$$A \to aA'$$
$$A' \to A' \mid \epsilon$$

ma viene rigenerata una nuova ricorsione sinistra immediata ($A' \to A'$)! L'algoritmo per rimuovere la ricorsione sinistra immediata non funziona se si applica ad una **produzione unitaria** ricorsiva sinistra (come $A \to A$, del tutto insensata!).

Esercizio riassuntivo

Togliere, nell'ordine,

1. simboli inutili
2. ricorsione sinistra
3. ϵ-produzioni
4. produzioni unitarie

dalla grammatica G:

$$S \to AB \mid D$$
$$A \to \epsilon \mid Ab \mid aB$$
$$B \to b \mid bB$$
$$C \to c$$
$$D \to aD$$

N.B.: L'ordine non è quello canonico (negli esercizi, quasi mai lo seguo).

Esercizio riassuntivo - 1

1. Elimino i simboli inutili:
   1.1 D non è generatore,
   1.2 C non è raggiungibile.

Ottengo G', senza simboli inutili, definita come:

$$S \to AB$$
$$A \to \epsilon \mid Ab \mid aB$$
$$B \to b \mid bB$$

Esercizio riassuntivo - 2

2. Elimino la ricorsione sinistra immediata $A \to Ab$. Ottengo G'', senza ricorsione sinistra, definita come:

$$S \to AB$$
$$A \to A' \mid aBA'$$
$$A' \to bA' \mid \epsilon$$
$$B \to b \mid bB$$

Esercizio riassuntivo - 3

3. Elimino le ϵ-produzioni.
L'insieme dei simboli annullabili è $N(G'') = \{A, A'\}$.
Ottengo G''', senza ϵ-produzioni, equivalente a G'', definita come:

$$S \to AB \mid B$$
$$A \to A' \mid aBA' \mid aB$$
$$A' \to bA' \mid b$$
$$B \to b \mid bB$$

N.B.: G''' è equivalente a G'' perché $\epsilon \notin L(G'')$.

Esercizio riassuntivo - 4

4. Elimino le produzioni unitarie: $U(G''') = Id \cup \{(S, B), (A, A')\}$.

Ottengo alla fine la grammatica:

$$S \to AB \mid b \mid bB$$
$$A \to bA' \mid b \mid aBA' \mid aB$$
$$A' \to bA' \mid b$$
$$B \to b \mid bB$$

Fattorizzazione a sinistra

Fattorizzazione a sinistra

Tecnica importante per parser top-down. Supponiamo di avere

$$A \to aBbC \mid aBd$$

Se, in top-down parsing, sulla pila ho A e leggo in input a, non sono in grado di determinare quale produzione scegliere (nondeterminismo).

Raccolgo la parte comune ("aB") alle 2 produzioni e introduco un nuovo nonterminale per rappresentare il "resto":

$$A \to aBA'$$
$$A' \to bC|d$$

Ora, con un solo simbolo di look-ahead, posso scegliere deterministicamente la produzione da espandere.

Fattorizzazione a sinistra: algoritmo

- Inizializza $N = NT$
- Ripeti il ciclo presente finché nessuna modifca è più possibile a N o all'insieme di produzioni:
  - for each $A \in N$
    - sia α il prefisso più lungo comune alle parti destre di alcune produzioni di A:
    - if $\alpha \neq \epsilon$
      - sia A' un nuovo nonterminale
      - $N := N \cup \{A'\}$
      - rimpiazza tutte le produzioni $A \rightarrow \alpha\beta_1|...|\alpha\beta_k|\gamma_1|...\gamma_h$ con $A \rightarrow \alpha A'|\gamma_1|...|\gamma_k$ e $A' \rightarrow \beta_1|...|\beta_k$

Esempio

$$E \to T \mid T + E \mid T - E$$
$$T \to A \mid A * T$$
$$A \to a \mid b \mid (E)$$

Si può fattorizzare:

$$E \to TE'$$
$$E' \to \epsilon \mid +E \mid -E$$
$$T \to AT'$$
$$T' \to \epsilon \mid *T$$
$$A \to a \mid b \mid (E)$$

Vedremo che questa grammatica fattorizzata è di classe LL(1), mentre quella originale non lo è.